

NETWORK LATENCY TRACER & OUTAGE LOGGER

USER GUIDE

Version: 1.0

Prepared By: *Team Alpha*

Project: *Ministry of ICT Internship Project*

Date: *July 2026*

Table of Contents

1. Introduction
2. Purpose of the System
3. Intended Users
4. System Requirements
5. System Architecture
6. Installation Guide
7. Starting the Application
8. Dashboard Overview
9. Managing Monitoring Hosts
10. Network Monitoring
11. Network Outage Detection
12. Viewing Analytics and Historical Data
13. Reports
14. Application Settings
15. Database Information
16. Troubleshooting
17. Frequently Asked Questions
18. Best Practices
19. Conclusion

1. Introduction

The **Network Latency Tracer & Outage Logger** is a network monitoring application designed to continuously monitor network hosts, measure latency performance, detect outages, and maintain historical records of network availability.

The system collects ICMP ping data from configured hosts, stores monitoring results in an SQLite database, analyses network performance, and presents information through a modern web dashboard.

The application consists of:

- A *FastAPI backend* responsible for monitoring, data processing, analytics, and API services.
- A *React frontend dashboard* responsible for displaying network statistics and allowing user interaction.
- An *SQLite database* responsible for storing latency records, outage information, and monitored hosts.

This project was developed as part of the **Ministry of ICT Internship Framework**.

2. Purpose of the System

The main objectives of the system are:

- Monitor network latency continuously.
- Detect network failures automatically.
- Record historical network performance.
- Track uptime and availability.
- Analyse network reliability.
- Provide visual monitoring dashboards.
- Generate performance reports.
- Assist administrators in troubleshooting network problems.

3. Intended Users

The application is designed for:

- Network Administrators
- IT Support Technicians
- System Administrators
- ICT Departments
- Students studying networking and system administration
- Researchers analysing network reliability

4. System Requirements

Hardware Requirements

Recommended minimum specifications:

- Processor: Dual-Core 2.0 GHz or higher
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 500 MB available space
- Active network connection

Software Requirements

Backend

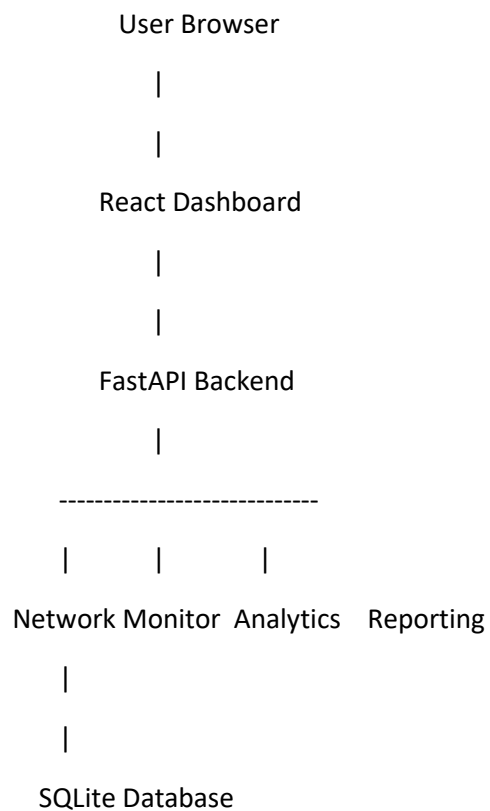
- Windows 10/11
- Python 3.11 or later
- FastAPI
- Uvicorn
- SQLite

Frontend

- Node.js
- npm
- Modern web browser:
 - Google Chrome
 - Microsoft Edge
 - Mozilla Firefox

5. System Architecture

The system follows a client-server architecture.



6. Installation Guide

Step 1: Download the Project

Clone the repository:

```
```bash
```

git clone <https://github.com/YourRepository/NetworkLatencyTracer.git>

Navigate into the project:

```
``bash
```

```
cd NetworkLatencyTracer
```

## Backend Installation

Navigate to the backend folder:

```
``bash
```

```
cd backend
```

Install required Python packages:

```
``bash
```

```
pip install -r requirements.txt
```

## Frontend Installation

Open another terminal.

Navigate to the frontend folder:

```
``bash
```

```
cd frontend
```

Install dependencies:

```
``bash
```

```
npm install
```

## 7. Starting the Application

### Start Backend Server

From the backend directory:

```
``bash
```

```
uvicorn api.main:app --reload
```

The API will start at:

```
http://127.0.0.1:8000
```

### Start Frontend Dashboard

From the frontend directory:

```
``bash
```

```
npm run dev
```

The dashboard will open at:

```
http://localhost:5173
```

## 8. Dashboard Overview

The dashboard provides a complete overview of network health.

### Summary Cards

The dashboard displays:

#### Total Hosts

Shows the number of monitored hosts.

Example:

14 Hosts

### **Fleet Uptime**

Displays the overall availability percentage.

Example:

99.5%

### **Average Latency**

Shows the average response time across monitored hosts.

Example:

45.6 ms

### **Total Outages**

Displays the number of detected network failures.

### **Total Downtime**

Shows the accumulated outage duration.

### **Performance Monitoring**

Users can select individual hosts and view:

- Latency trends
- Packet loss trends
- Availability statistics
- Outage history

## 9. Managing Monitoring Hosts

Users can add and manage monitoring targets through the dashboard.

A host requires:

- Host name
- IP address

Example:

Name:

Google DNS

Address:

8.8.8.8

Available actions:

- Add host
- Remove host
- Enable monitoring
- Disable monitoring

After adding a host, the monitoring engine automatically detects the new target and begins collecting latency data.

## 10. Network Monitoring

The monitoring engine performs continuous ICMP checks.



For every host, the system records:

- Timestamp
- Host address
- Ping sequence number
- Response status
- Latency value
- Error information

Example:

Host:

8.8.8.8

Latency:

35 ms

Status:

Online

Monitoring runs automatically while the backend service is active.

## 11. Network Outage Detection

The system automatically detects network failures.

An outage is triggered when multiple consecutive ping requests fail.

The system records:

- Host affected

- Outage start time
- Recovery time
- Total outage duration
- Failure reason

Example:

Host:

Google DNS

Start:

10:32:15

End:

10:36:45

Duration:

270 seconds

## 12. Viewing Analytics and Historical Data

The analytics module processes stored monitoring information.

Users can view:

- Average latency
- Minimum latency
- Maximum latency
- Packet loss percentage
- Uptime percentage
- Reliability rating
- MTTR

- MTBF

Historical data helps identify:

- Network instability
- Frequent outages
- Performance degradation

### 13. Reports

The application supports report generation.

Available formats:

- CSV
- PDF

Reports contain:

- Monitoring period
- Total hosts monitored
- Average latency
- Uptime percentage
- Packet loss
- Total outages
- Downtime statistics
- Host performance ranking

Reports can be generated from the dashboard.

### 14. Application Settings

The monitoring behaviour can be configured.

### **Monitoring Interval**

Controls how often ping requests are sent.

Example:

5 seconds

### **Ping Timeout**

Defines how long the system waits for a response.

Example:

2 seconds

### **Failure Threshold**

Defines the number of failed requests required before an outage is detected.

Example:

3 failures

## **15. Database Information**

The application uses SQLite for data storage.

Database location:

database/latency.db

Main database tables:

## **Hosts**

Stores monitored targets.

Contains:

- Host name
- IP address
- Status
- Creation date

## **Latency Logs**

Stores:

- Host
- Timestamp
- Ping result
- Latency
- Error information

## **Outage Logs**

Stores:

- Host
- Start time
- End time
- Duration
- Reason

---

## **# 16. Troubleshooting**

### **## Backend Does Not Start**

Possible solutions:

- Confirm Python is installed.
- Install required packages.
- Check the terminal location.
- Verify the correct command:

'''

```
uvicorn api.main:app --reload
```

Frontend Does Not Start

**Possible solutions:**

- Confirm Node.js is installed.
- Run:

```
npm install
```

- Start:

```
npm run dev
```

No Latency Data Appears

**Possible solutions:**

- Confirm the monitoring service is running.
- Check the host address.
- Verify network connectivity.
- Confirm the host responds to ping.

## Database Errors

Possible solutions:

- Confirm the database folder exists.
- Restart the backend.
- Check database permissions.

## 17. Frequently Asked Questions

### **Can multiple hosts be monitored?**

Yes. The system supports monitoring multiple network targets simultaneously.

### **Does monitoring continue automatically?**

Yes. Once the backend monitoring service starts, hosts are monitored automatically.

### **Where is monitoring data stored?**

All monitoring information is stored in:

database/latency.db

### **Can reports be exported?**

Yes. Reports can be exported as CSV and PDF files.

### **Can hosts be added after starting monitoring?**

Yes. Newly added hosts are detected automatically by the monitoring service.

## 18. Best Practices

For optimal operation:

- Monitor reliable hosts.
- Regularly export reports.
- Backup the SQLite database.
- Review outage history.

- Remove unused hosts.
- Keep dependencies updated.
- Monitor available disk space.

## 19. Conclusion

The Network Latency Tracer & Outage Logger provides an effective solution for monitoring network performance and reliability.

Through continuous latency measurement, outage detection, analytics, reporting, and dashboard visualization, the system assists administrators in identifying network issues and improving service availability.

This guide provides the necessary information required to in